Segurança em Redes de Comunicações

High-Availability Firewalls Scenarios

iptables (nf_tables)

If the local/routerfw docker image is not available in your setup, do:

With the provided Dockerfile, create a custom docker image:

```
$ docker build -t local/routerfw -f Dockerfile_routerfw .
```

Add the Docker container to GNS3 as template:

- Choosing an existing image;
- Define it as local/routerfw container image;
- Activate 8 network interfaces (adapters), and
- **Add the following directories as permanent in the advanced tab:**

/etc/frr/
/etc/conntrackd
/etc/init.d/

Note: FRRouting image is a linux container with a set of services/protocols installed that enable advance network features. The command vtysh provides a Cisco-like CLI.

If the local/pcclient docker image is not available in your setup, do:

With the provided Dockerfile, create a custom docker image:

```
$ docker build -t local/pclient -f Dockerfile_pcclient .
```

Add the Docker container to GNS3 as template:

- Choosing an existing image;
- Define it as local/pcclient container image;
- Activate 1 network interface (adapter), and
- **Add the following directories as permanent in the advanced tab:**

/etc/wpa_supplicant/
/etc/init.d/

Note: For both docker images, the Linux bash commands are not permanent! They will disappear upon a reboot. To make them permanent write them in file **/etc/init.d/adv_network**. These commands will be run on every start of the
Docker container.

Note2: To save iptables rules using:

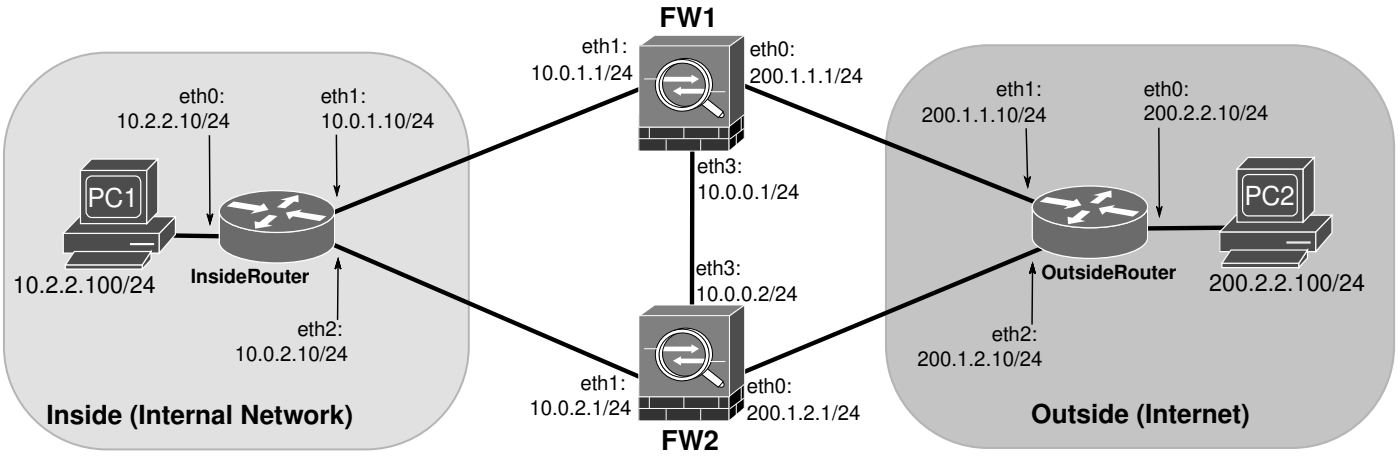
```
iptables-save > /etc/init.d/iptables.rules
```

To restore iptables rules use:

```
iptables-restore < /etc/init.d/iptables.rules
```

Add the restore command to **/etc/init.d/adv_network**

Active-Active Scenarios



Asymmetric Routing and Firewall Synchronization

2. Configure the network depicted in the previous figure using GNS3 with PC1 and PC2 as as local/pcclient docker, and the Load Balancers, Routers, and Firewalls as a local/routerfw docker. Configure PCs addresses and gateways, and Routers' addresses and static routes.

For testing purposes configure asymmetric routing: InsideRouter routes via FW2, and OutsideRouter routes via FW1.

You must assume that the Internal Network has the 192.1.0.0/23 IPv4 public network.

```
InsideRouter$ vtysh
InsideRouter# configure
InsideRouter(config)# interface eth0
InsideRouter(config-if)# ip address 10.2.2.10/24
InsideRouter(config-if)#no shutdown
InsideRouter(config-if)#interface eth1
InsideRouter(config-if)#ip address 10.0.1.10/24
InsideRouter(config-if)#no shutdown
InsideRouter(config-if)#interface eth2
InsideRouter(config-if)#ip address 10.0.2.10/24
InsideRouter(config-if)#no shutdown
InsideRouter(config)#ip route 0.0.0.0/0 10.0.2.1
–
OutsideRouter$ vtysh
OutsideRouter# configure
OutsideRouter(config)# interface eth0
OutsideRouter(config-if)# ip address 200.2.2.10/24
OutsideRouter(config-if)# no shutdown
OutsideRouter(config-if)# interface eth1
OutsideRouter(config-if)# ip address 200.1.1.10/24
OutsideRouter(config-if)# no shutdown
OutsideRouter(config-if)# interface eth2
OutsideRouter(config-if)# ip address 200.1.2.10/24
OutsideRouter(config-if)# no shutdown
OutsideRouter(config)# ip route 192.1.0.0/23 200.1.1.1
```

For initial testing (before NAT configuration in FW1 and FW2) add also a static route to the private networks:

```
OutsideRouter(config)# ip route 10.0.0.0/8 200.1.1.1
```

>> Verify the Routers routing tables with sh ip fib

2. Configure the firewalls IPv4 addresses and basic static routing using the following commands:

- For FW1:

```
# ip addr add 200.1.1.1/24 dev eth0
# ip addr add 10.0.1.1/24 dev eth1
# ip addr add 10.0.0.1/24 dev eth3
# ip route add 0.0.0.0/0 via 200.1.1.10
# ip route add 10.2.2.0/24 via 10.0.1.10
```

- For FW2:

```
# ip addr add 200.1.2.1/24 dev eth0
# ip addr add 10.0.2.1/24 dev eth1
# ip addr add 10.0.0.2/24 dev eth3
# ip route add 0.0.0.0/0 via 200.1.2.10
# ip route add 10.2.2.0/24 via 10.0.2.10
```

>> Verify the configured addresses with: ip addr

>> Verify the Firewalls' routing tables with: ip route

>> Test the full connectivity between PC1 and PC2.

Do not proceed if PC1 and PC2 do not have connectivity.

3. Configure the firewalls NAT/PAT mechanisms:

- For FW1 and FW2:

```
# iptables -t nat -A POSTROUTING -j SNAT -o eth0 -s 10.0.0.0/8 --to 192.1.0.1-192.1.0.10
```

>> Test the full connectivity between PC1 and PC2 using UDP pings:

```
nping --udp 200.2.2.100 -p 5001
```

>> Use the following command to verify the active NAT translations in both Firewalls: conntrack -L

>> Explain the lack of connectivity.

4. Configure the firewalls high-availability connection tracking/state synchronization (conntrackd) mechanism. See conntrackd manual in https://conntrack-tools.netfilter.org/manual.html .

To configure conntrackd in FW1:

Edit /etc/conntreckd/contrackd.conf, by **removing** section **Stats** and **adding** section **Sync**:

```
Sync {
        Mode FTFW {
                DisableExternalCache On
        }
        Multicast {
                IPv4_address 225.0.0.50
                Group 3780
                IPv4_interface 10.0.0.1
                Interface eth3
                SndSocketBuffer 8388608
                RcvSocketBuffer 8388608
                Checksum on
        }
}
```

For FW2, repeat the same procedure, with a different IPv4_interface configuration:

```
Sync {
        ...
                IPv4_interface 10.0.0.2
        ...
}
```

Kill and restart the conntrackd service (both in FW1 and FW2) with:

```
 # service conntrackd restart
```

>> Capture packets in the link between FW1 and FW2 and analyze/identify the exchanges packets.

>> Test the full connectivity between PC1 and PC2 using UDP pings:

```
        nping --udp 200.2.2.100 -p 5001
```

>> Use the following command to verify the active NAT translations in both Firewalls

```
 $ conntrack -L
```

>> Use the following commands to analyze the current tracked and synced connections:

```
 $ conntrackd -i
 $ conntrackd -s
```

>> Explain why now there is connectivity between the PCs.

>> Explain the advantage of using, for synchronization, a multicast address (225.0.0.50) compared to a unicast address.

Note: The command "DisableExternalCache On" forces an immediate use of the states received from the other Firewall, enables the Active-Active scenario.

5. Configure a flow control rule in FW1 and FW2 to allow only UDP traffic (from Inside to Outside zones) that use UDP destination ports between 5000 and 5005.

Define the default policy for FORWARD chain to DROP, create the Firewall zones INSIDE and OUTSIDE and create chains FROM-INSIDE-TO-OUTSIDE and FROM-OUTSIDE-TO-INSIDE with the respective flow control policy, using following configuration **on both Firewalls**:

```
# iptables -P FORWARD DROP
# iptables -N ZONE-INSIDE
# iptables -A FORWARD -o eth1 -j ZONE-INSIDE
# iptables -N ZONE-OUTSIDE
# iptables -A FORWARD -o eth0 -j ZONE-OUTSIDE

# iptables -A ZONE-INSIDE -i eth1 -j RETURN
# iptables -A ZONE-OUTSIDE -i eth0 -j RETURN

# iptables -N FROM-OUTSIDE-TO-INSIDE
# iptables -A ZONE-INSIDE -i eth0 -j FROM-OUTSIDE-TO-INSIDE

# iptables -N FROM-INSIDE-TO-OUTSIDE
# iptables -A ZONE-OUTSIDE -i eth1 -j FROM-INSIDE-TO-OUTSIDE

# iptables -A FROM-INSIDE-TO-OUTSIDE -p udp --dport 5000:5005 -j ACCEPT

# iptables -A FROM-OUTSIDE-TO-INSIDE -m state --state RELATED,ESTABLISHED -j ACCEPT
```

>> Test the connectivity from PC1 to PC2 using UDP pings, and multiple ports on the range 5000-5005:

```
nping --udp 200.2.2.100 -p 5001
nping --udp 200.2.2.100 -p 5002
...
```

>> Analyze the connectivity test results.

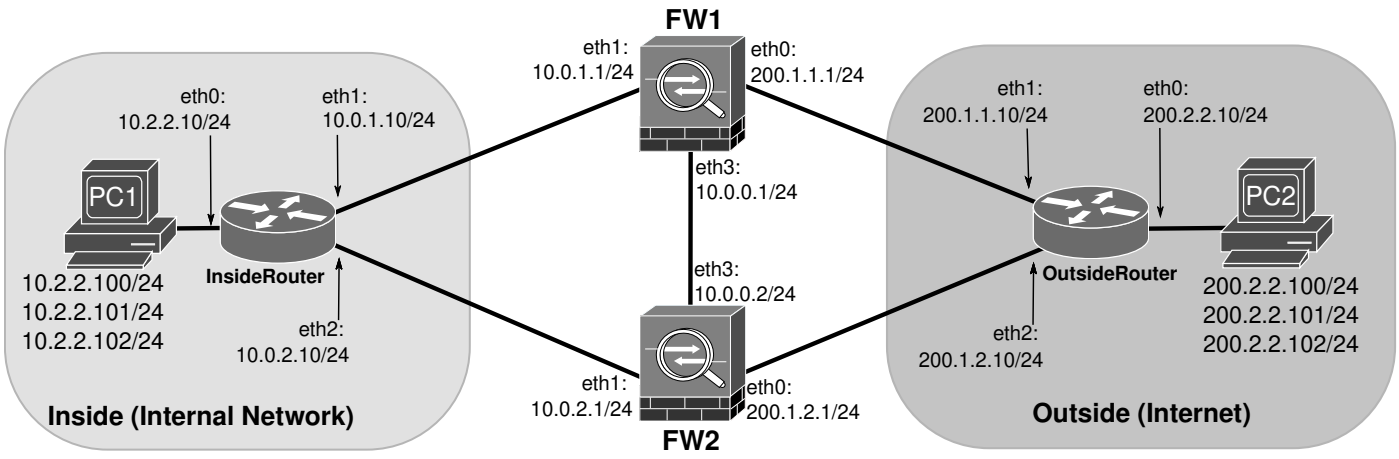
>> Using Wireshark, identify which Firewall is being chosen by the load balancer for each data flow.

>> Verify the correct synchronization of the flow states between both firewalls.

>> Explain why the asymmetric routing solution is not correct and do not implement a fully functional High-Availability Firewalls.

Linux Default Routing (local decision hash based load balancing, with firewall synchronization)

Stop the GNS3 project, and rename it.



6. Using the same network as before, reconfigure the routers to have both possible routes (via FW1 or FW2) with the same weights on the routing table.

```
InsideRouter$ vtysh
InsideRouter# configure
InsideRouter(config)# ip route 0.0.0.0/0 10.0.1.1
InsideRouter(config)# ip route 0.0.0.0/0 10.0.2.1
–
OutsideRouter$ vtysh
OutsideRouter# configure
OutsideRouter(config)# ip route 192.1.0.0/23 200.1.1.1
OutsideRouter(config)# ip route 192.1.0.0/23 200.1.2.1
```

Add additional IPv4 addresses to PC1 and PC2:

```
PC1# ip addr add 10.2.2.101/24 dev eth0
PC1# ip addr add 10.2.2.102/24 dev eth0
PC2# ip addr add 200.2.2.101/24 dev eth0
PC2# ip addr add 200.2.2.102/24 dev eth0
```

Start two Wireshark captures, in links InsideRouter-FW1 and InsideRouter-FW2.

Test connectivity from PC1 to PC2 using UDP pings, and multiple ports on the range 5000-5005:

```
nping --udp 200.2.2.100 -p 5001
nping --udp 200.2.2.100 -p 5002
```

Test connectivity from PC1 to the new PC2 IPv4 addresses using UDP pings:

```
nping --udp 200.2.2.101 -p 5001
nping --udp 200.2.2.102 -p 5001
```

Test connectivity from PC1 to PC2 (all IPv4 addresses), using different source IPv4 addresses:

```
nping --udp 200.2.2.100 -p 5001 -S 10.2.2.101
nping --udp 200.2.2.100 -p 5001 -S 10.2.2.101
```

Test connectivity from PC1 to PC2 with multiple combinations of source and destination address, and destination port.

>> Using Wireshark, identify which Firewall is being chosen by the Inside Router for each data flow.

>> Using Wireshark, identify which Firewall is being chosen by the Outside Router for each data flow. Does asymmetric routing can still exist?

>> Verify the correct synchronization of the flow states between both firewalls (conntrackd must be active).

>> Identify the percentage of flows using each Firewall.

>> How the default Load Balancing works in terms of choosing a different path (in terms of source and destination addresses and destination port).

Note: At the Linux kernel level, the routing process is a hash based process on a per-flow decision (differentiated by source/destination IP addresses). Upon the reception of a new inbound packet the route next-hop is strictly defined by a hash function with an initial random seed, this means that after reboot decision may change for same flow.

7. Using the same network, remove the direct connection between the Firewalls, disable the synchronization and clear conntrackd table **on both Firewalls**:

```
# service conntrackd stop
```

>> Test the connectivity from PC1 to PC2 using UDP pings, and multiple source and destination addresses, and ports on the range 5000-6000:

```
nping --udp 200.2.2.100 -p 5001 -S 10.2.2.101
nping --udp 200.2.2.101 -p 5002 -S 10.2.2.100
```

>> Which connectivity tests still have response?

>> The possible asymmetric routing have any impact when firewalls do not synchronize?

(Optional) In **all Routers and Firewalls**, activate BFD (Bidirectional Forwarding Detection) and define peers:

```
InsideRouter(config)# bfd
InsideRouter(config-bfd)# peer 10.0.1.1
InsideRouter(config-bfd)# peer 10.0.1.2
--
OutsideRouter(config)# bfd
OutsideRouter(config-bfd)# peer 200.1.1.1
OutsideRouter(config-bfd)# peer 200.1.2.1
--
FW1(config)# bfd
FW1(config-bfd)# peer 10.0.1.10
FW1(config-bfd)# peer 200.1.1.10
--
FW2(config)# bfd
FW2(config-bfd)# peer 10.0.2.10
FW2(config-bfd)# peer 200.1.2.10
```

In **both routers**, redefine the static routers to activate BFD:

```
InsideRouter(config)# no ip route 0.0.0.0/0 10.1.1.1
InsideRouter(config)# no ip route 0.0.0.0/0 10.1.1.2
InsideRouter(config)# ip route 0.0.0.0/0 10.1.1.1 bfd
InsideRouter(config)# ip route 0.0.0.0/0 10.1.1.2 bfd
–
OutsideRouter(config)# no ip route 192.1.0.0/23 200.1.1.1
OutsideRouter(config)# no ip route 192.1.0.0/23 200.1.1.2
OutsideRouter(config)# ip route 192.1.0.0/23 200.1.1.1 bfd
OutsideRouter(config)# ip route 192.1.0.0/23 200.1.1.2 bfd
```

Start a Wireshark capture in some of the links between the Routers and Firewalls. Verify the routing tables in both routers. Stop Firewall FW1. Re-verify the routing tables in both routers. Restart Firewall FW1. Re-verify the routing tables in both routers.

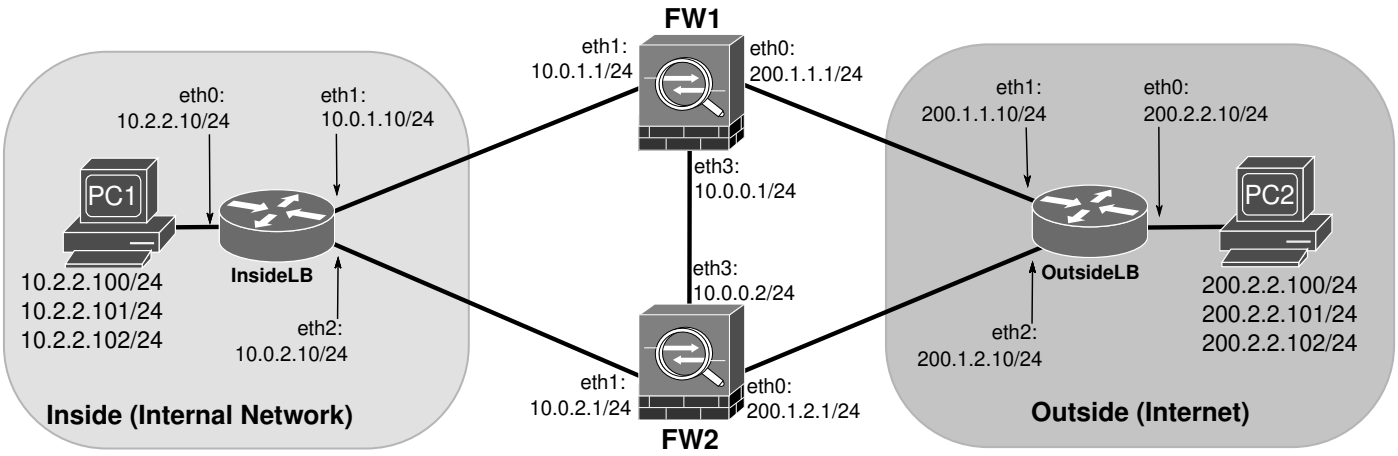
>> What can you conclude about the functionality of BFD protocol?

>> What can you conclude about the routing/load balancing when a firewalls fails?

Note: This commands require that bfdd daemon must be running in both Routers and Firewalls.

Probabilistic/Asymmetric Load Balancing (stateful load balancing, without firewall synchronization)

Stop the GNS3 project, and rename it.



8. Using the same network as before. Kill the conntrackd service (both in FW1 and FW2) with:

```
# service conntrackd stop
```

Remove from both Routers/load balancers all configured static routes:

```
InsideLB$ vtysh
InsideLB# configure
InsideLB(config)# no ip route 0.0.0.0/0 10.0.1.1
InsideLB(config)# no ip route 0.0.0.0/0 10.0.2.1
–
OutsideLB$ vtysh
OutsideLB# configure
OutsideLB(config)# no ip route 192.1.0.0/23 200.1.1.1
OutsideLB(config)# no ip route 192.1.0.0/23 200.1.2.1
```

>> Confirm that services connectivity has been lost.

>> Verify the routing tables with: ip route

Fully disable the Reverse Path Validation (rp_filter). Reverse path validation checks that incoming packets have a valid return route (on the main routing table) back through the same interface they arrived on. This is not true with custom routing tables.

In both load balancers:

```
sysctl -w net.ipv4.conf.all.rp_filter=0
sysctl -w net.ipv4.conf.default.rp_filter=0
sysctl -w net.ipv4.conf.eth0.rp_filter=0
sysctl -w net.ipv4.conf.eth1.rp_filter=0
sysctl -w net.ipv4.conf.eth2.rp_filter=0
```

9. Using iptables and custom routing tables, implement an asymmetric load-balancing (70% of flows via FW1). To configure the Load-Balanceres load balancing service, the iptables **mangle** table must be used to mark packets/connections (before routing, using the PREROUTING chain) on the connections tracking service to "memorize" routes. For new connections a random mark is used to define a route for that flow. The packets from marked connections will use custom routing tables with specific static routes. To implement such solution use the following commands **identifying the purpose and logic of each set of commands**:

- For InsideLB and OutsideLB (interfaces names must be the same on both LB):

```
# iptables -t mangle -N IN_eth1
# iptables -t mangle -A IN_eth1 -j CONNMARK --set-mark 101
# iptables -t mangle -N IN_eth2
# iptables -t mangle -A IN_eth2 -j CONNMARK --set-mark 102

# iptables -t mangle -N OUT_eth1
# iptables -t mangle -A OUT_eth1 -j CONNMARK --set-mark 101
# iptables -t mangle -A OUT_eth1 -j MARK --set-mark 101
# iptables -t mangle -A OUT_eth1 -j ACCEPT

# iptables -t mangle -N OUT_eth2
# iptables -t mangle -A OUT_eth2 -j CONNMARK --set-mark 102
# iptables -t mangle -A OUT_eth2 -j MARK --set-mark 102
# iptables -t mangle -A OUT_eth2 -j ACCEPT

# iptables -t mangle -N LOADBALANCE
# iptables -t mangle -A LOADBALANCE -i eth0 -m state --state NEW -m statistic --mode random --probability 0.7 -j OUT_eth1
# iptables -t mangle -A LOADBALANCE -i eth0 -m state --state NEW -j OUT_eth2
# iptables -t mangle -A LOADBALANCE -i eth0 -j CONNMARK --restore-mark

# iptables -t mangle -A PREROUTING -i eth1 -m state --state NEW -j IN_eth1
# iptables -t mangle -A PREROUTING -i eth2 -m state --state NEW -j IN_eth2
# iptables -t mangle -A PREROUTING -j LOADBALANCE
```

- For InsideLB

```
# ip rule add fwmark 101 table 101
# ip rule add fwmark 102 table 102
# ip route add default via 10.0.1.1 table 101
# ip route add default via 10.0.2.1 table 102
```

- For OutsideLB:

```
# ip rule add fwmark 101 table 101
# ip rule add fwmark 102 table 102
# ip route add 192.1.0.0/23 via 200.1.1.1 table 101
# ip route add 192.1.0.0/23 via 200.1.2.1 table 102
```

Verify the correct configuration of the load-balance process with:

```
# iptables -t mangle -L -vn
# ip rule
# ip route
# ip route list table 101
# ip route list table 102
```

Note: 0x65=101, 0x66=102

>> Explain how iptables (mangle table) marks packets/connections (input and output), and how packet marks determined the routing.

>> Test the connectivity from PC1 to PC2 using UDP pings, and multiple ports on the range 5000-5005:

```
nping --udp 200.2.2.100 -p 5002
```

>> Start packet captures on the links that connect the firewalls and load balancers, and verify the correctness of the load balancing process between FW1 and FW2.

>> Identify the percentage of packets using each Firewall.

>> Why not all packets from the same flow use the same path?

>> Inspect the status and active connections of the load balancing process:

```
# conntrack -L
```

>> Why conntrack change the mark associated with each flow over time (when new packet arrive)?

>> Use socat client/server to test fully completed (ASSURED) connections. What can you conclude about the connections marks and routing decisions for all packets of the same flow?

PC2 server UDP 5001:

```
socat -d2 UDP-LISTEN:5001,reuseaddr,fork SYSTEM:'while read msg; do echo "Server reply: $msg"; done' &
```

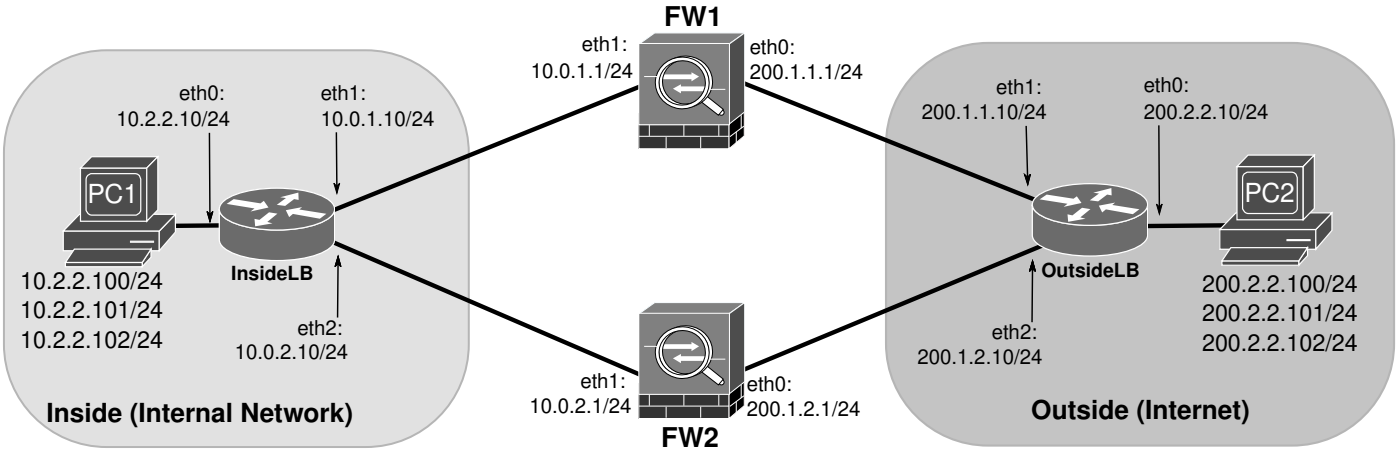
PC1 client: `socat – UDP:200.2.2.100:5001`

>> Can redundant Load Balancers be installed without synchronizing conntrack states/marks?

Note: The load balancing process is a stateful process on a per-flow decision. Upon the reception of a new inbound flow, a route is randomly chosen, and the choice is kept on the connection state table, after fully established. This approach is not optimal for DDoS protection, ideally the decision should be based on a hash function (dependent of flow parameters - IP and/or ports) with no state kept in memory.

Hash Function based Load Balancing (stateless load balancing, without firewall synchronization)

Stop the GNS3 project, and rename it.



Guarantee that Reverse Path Validation (rp_filter) is fully disabled. **In both load balancers:**

```
sysctl -w net.ipv4.conf.all.rp_filter=0
sysctl -w net.ipv4.conf.default.rp_filter=0
sysctl -w net.ipv4.conf.eth0.rp_filter=0
sysctl -w net.ipv4.conf.eth1.rp_filter=0
sysctl -w net.ipv4.conf.eth2.rp_filter=0
```

10. Using the same network as before, remove the link between FW1 and FW2. Delete all rules from mangle iptables from InsideLB and OutsideLB:

```
# iptables -t mangle -F
```

Configure the Load Balancing with iptables to mark packets/connections based on the **OUTSIDE address and port**.

For InsideLB:

```
# iptables -t mangle -N LOADBALANCE
# iptables -t mangle -A LOADBALANCE -i eth0 -j HMARK --hmark-rnd 1 --hmark-tuple dst,dport --hmark-mod 2 --hmark-offset 101
#iptables -t mangle -A PREROUTING -j LOADBALANCE
```

For OutsideLB:

```
# iptables -t mangle -N LOADBALANCE
# iptables -t mangle -A LOADBALANCE -i eth0 -j HMARK --hmark-rnd 1 --hmark-tuple src,sport --hmark-mod 2 --hmark-offset 101
# iptables -t mangle -A PREROUTING -j LOADBALANCE
```

>> Use socat client/server to test fully completed connections. What can you conclude about the connections marks and routing decisions for all packets of the same flow?

PC2 server UDP 5001:

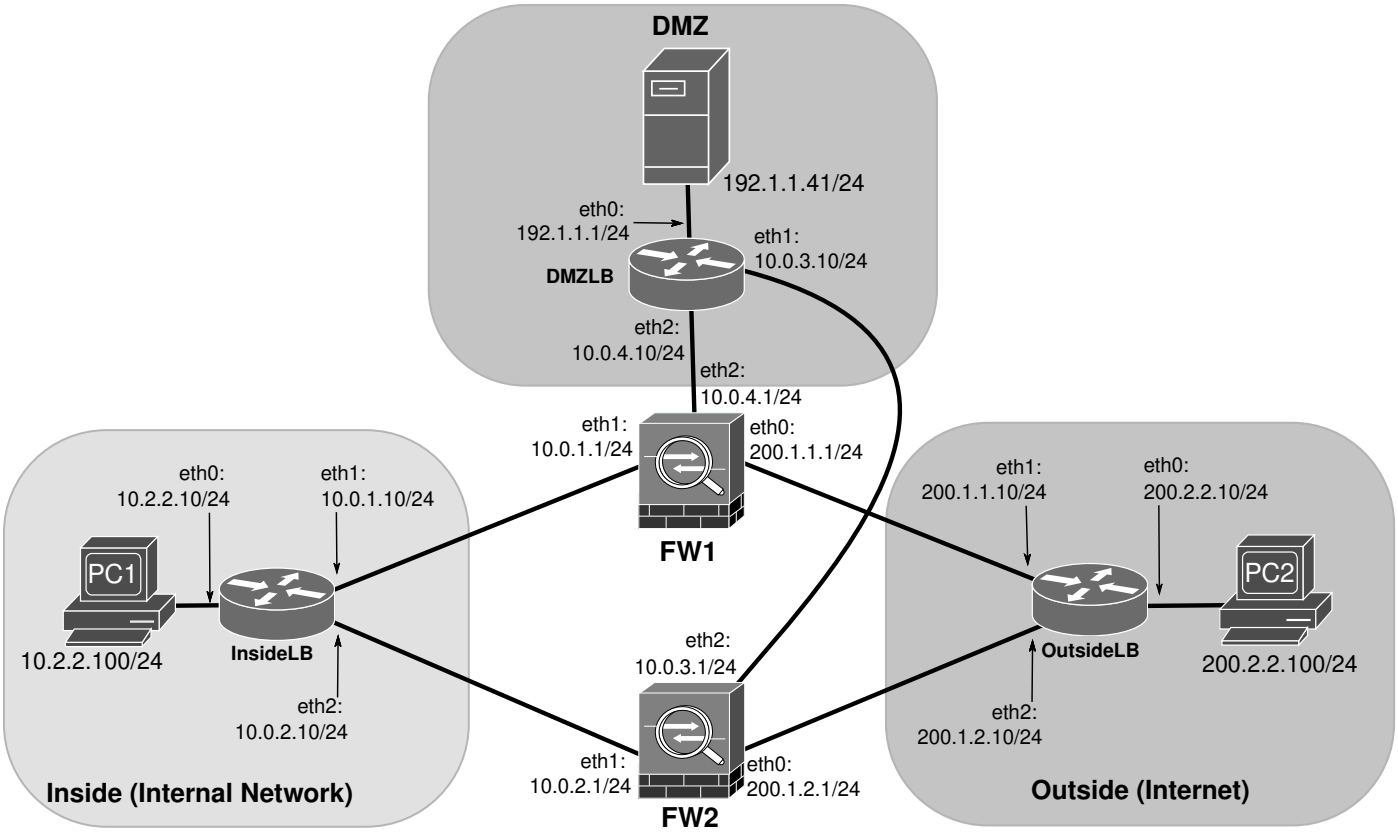
```
socat -d2 UDP-LISTEN:5001,reuseaddr,fork SYSTEM:'while read msg; do echo "Server reply: $msg"; done' &
```

PC1 client: socat – UDP:200.2.2.100:5001

>> What are the advantages of using a fully stateless setup on both Firewalls and Load Balancers?

>> Can redundant Load Balancers be installed without synchronizing conntrack states/marks? What are the constrains/requirements?

Policies Definition and Integrated Deployment



11. To the previous setup add a DMZ and deploy a server (using the local/server docker image) that supports multiple services (e.g., HTTP, HTTPS, DNS, SSH, etc…) that can be simulated with the socat command.

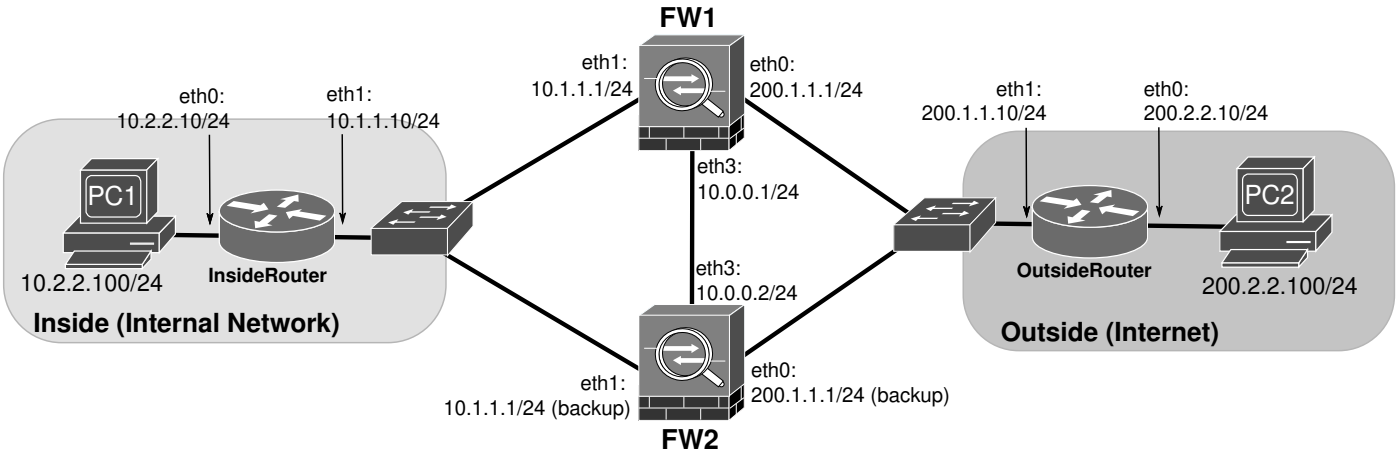
>> Define a set of flow control policies that (i) incorporate the good practices of network defense, (ii) allow the internal users a limited access to internal/DMZ and external services, (iii) allow strict access to the public services available on the DMZ server(s), and (iv) block external users during a possible DDoS attack.

You may assume that the OutsideLB may work as a stateless firewall. In a real scenario the load balancers and stateless firewalls should be different devices.

>> Deploy the appropriate firewall rules in the firewalls. You may assume that exists an external monitoring system that will identify the external IP address of the DDoS participants and dynamically provide an updated list.

>> (Extra) Create a script (running in a internal server directly connected to the firewalls) that automatically creates the blocking rules in the firewalls during a DDoS attack upon the identification of the attackers IP addresses. Suggestion: consider a remote connection via SSH to the firewalls and/or load balancers.

Extra (Optional) - Active-Backup Scenario (VRRP Protocol)



Assemble the above network. Configure all IPv4 addresses and Routes except fot interfaces eth0 and eth1 in both Firewalls. This interfaces will have Virtual IP addresses managed by keepalived using the VRRP protocol.

On both Firewalls, to enable two instances of VRRP (Inside and Outside networks), create the file /etc/init.d/keepalived.conf with the following contents:

```
vrrp_instance VI_1 {
    state MASTER
    interface eth1
    virtual_router_id 51
    priority 150
    advert_int 1
    unicast_src_ip 10.0.0.1
    unicast_peer {
        10.0.0.2
    }
    virtual_ipaddress {
        10.1.1.1/24
    }
    notify_master "/etc/init.d/ha_routes"
}

vrrp_instance VI_0 {
    state MASTER
    interface eth0
    virtual_router_id 52
    priority 150
    advert_int 1
    unicast_src_ip 10.0.0.1
    unicast_peer {
        10.0.0.2
    }
    virtual_ipaddress {
        200.1.1.1/24
    }
    notify_master "/etc/init.d/ha_routes"
}
```

On both Firewalls, create the file /etc/init.d/ha_routes with the following contents:

```
#!/bin/bash
/sbin/ip route add 0.0.0.0/0 via 200.1.1.10
/sbin/ip route add 10.2.2.0/24 via 10.1.1.10
```

On both Firewalls, start the keepalived daemon:

```
keepalived -f /etc/init.d/keepalived.conf
```

>> Test connectivity between PC1 and PC2.

>> Analyze the VRRP configuration file. Explain the purpose of the virtual_ipaddress section.

>> Capturing packets on the Firewall links, analyze the VRRP packets. Explain their purpose.

>> Stop the active firewall (FW1).

>> Re-Test connectivity between PC1 and PC2. Explain the FW2 transition process from backup to active.

>> Restart firewall F1. Explain the FW2 transition process from active to backup.

To stop keepalived: killall keepalived